

LLM PIPELINES & AI FDE WORKFLOWS

Study Guide — From a Business Problem to a Working Copilot

Week 2 · Day 2 · Prepared by Rajesh Pasham

Program Context

Item	Detail
Session Type	Concepts and workflow lecture — no live Palantir platform work today
Platform	Individual Palantir Developer Tier accounts, used from Day 3 onward
This Week's Lab (Day 3)	Build your batch's Copilot for real — today's workflow is exactly what you'll follow

Learning Objectives

By the end of today, you will be able to:

- Design a reusable LLM pipeline, not a single one-off prompt
- Prepare Ontology objects and documents for LLM consumption
- Explain Ontology-aware retrieval and how it differs from plain keyword retrieval
- Integrate Functions and tools into a pipeline the model can call
- Walk the AI Forward Deployed Engineering (FDE) workflow, from operational requirement to operational solution
- Design a Copilot's limitations and escalation path deliberately, before building it

1. Where Today Fits

Day 1 gave you the architecture vocabulary — tokens, prompt layers, grounding, safety. Today gives you the process: how a real operational requirement actually becomes a working Copilot, step by step, and what “done” looks like before you escalate to a human instead of guessing. This is the workflow every batch runs for real starting Day 3.

2. Designing Reusable LLM Pipelines

A single prompt answers one question well; a pipeline answers a family of questions. Your batch's Copilot needs to handle variation — different claims, applications, or cases — not just the one example you tested during design.

- Each stage should be independently testable — data preparation, retrieval, and prompt assembly are separate stages, debugged separately rather than as one opaque block
- Reusable means versioned and shared — the same prompt-template pattern from Day 1 applies here: fixed structure, varying content

3. The Pipeline, Stage by Stage

Stage	What It Does
Data Preparation	Getting Ontology objects and documents into a form an LLM can consume
Context Pipeline (Retrieval)	The retrieval step that decides what's relevant to this specific question
Prompt Assembly	Combining system, user, and contextual instructions — Day 1's three layers, now automated
Function / Tool Call	Where the pipeline stops retrieving and starts acting or computing

4. Data Preparation for LLM Consumption

Structured data: an Ontology object has many properties — decide which ones actually belong in the prompt, not all of them by default. Unstructured data: documents need the chunking and embedding pipeline (covered in depth in Week 4) before an LLM can retrieve from them meaningfully. The common mistake is passing an entire object or document verbatim “to be safe” — this is the same context-window cost problem from Day 1, just appearing earlier in the pipeline.

5. Prompt & Context Pipelines

The context pipeline decides what's relevant, per call: given a claimId or applicationId, what should actually be retrieved runs every time a question is asked, not once at design time. The prompt pipeline assembles the final compiled prompt automatically — system instructions (fixed) plus retrieved context (variable) plus the user's question — rather than being hand-typed per call.

6. Ontology-Aware Retrieval

Retrieval can follow Link Types, not just match text. Given a Claim, ontology-aware retrieval can pull its Policy and Policyholder automatically — the relationships you defined in Week 1 become retrieval logic. It's also governance-aware by construction: if a property is Marking-protected, ontology-aware retrieval respects that as part of how it works, not as a separate check bolted on afterward.

7. Function & Tool Integration

A tool is a capability the model can invoke, not just data it reads — a lookup Function, a calculation, or (from Week 3 onward) an Action. The model decides when to call a tool based on the tools you've exposed to it and how you've described them. Describe each tool clearly enough that the model knows when to use it: a vague tool description leads to a model either ignoring a useful tool or calling the wrong one.

8. The AI Forward Deployed Engineering (FDE) Workflow

A five-stage discipline for going from a vague ask to a scoped build:

Stage	What Happens
1	Operational Requirement — the problem as a business stakeholder describes it, in their own words
2	Translate to AIP Use Case — turning that requirement into a specific, buildable capability
3	Rapid Prototype — building the smallest thing that tests the real question
4	Test with Business Users — validating with the people who'll actually use it, not just engineers
5	Operational Solution — governed, reliable, and ready for production use

9. Translating Operational Requirements into AIP Use Cases

This is the step most teams rush, and shouldn't. The requirement as given — “adjusters spend too long reviewing claim documents” — is real, but not yet buildable. The translation asks: what specific question, asked of what specific Ontology objects, would actually save that time? Name it precisely. The output is a scoped Copilot capability: “Given a claimId, summarize the claim and flag missing information” — buildable, testable, and exactly what this week's Practical Lab specifies.

10. Rapid Prototyping

- Prototype the riskiest assumption first — if you're unsure whether grounding will actually work for this use case, test that before polishing the prompt wording
- A prototype is disposable by design — expect to rebuild parts of it once you see real usage; that's the point of prototyping quickly, not a sign it failed

11. Testing with Business Users

- Domain experts catch what engineers can't — an adjuster or underwriter will notice a wrong risk judgment that reads as perfectly plausible to someone outside the domain
- Watch them use it, don't just ask if it's good — what someone actually tries to ask the Copilot is more revealing than their opinion of a demo you walked them through
- Real usage surfaces the edge cases your test data didn't — this is where you'll find the ambiguous questions Day 1's "handling ambiguity" topic was preparing you for

12. Moving from Prototype to Operational Solution

Prototype	Operational Solution
Answers the question, for the cases you tested, with you watching	Answers the question reliably, for cases you didn't test, with governance, Evals, and observability in place

The gap between them is everything this program builds from Week 3 onward — writeback, document intelligence, multi-agent design, and production readiness all exist to close exactly this gap.

13. Building Initial Functional Copilots

What "functional" actually requires, per this week's Practical Lab specification:

- Accepts an identifier and retrieves grounded data — a claimId, applicationId, fraudCaseId, or policyholderId, the entry point for your batch's Copilot
- Summarizes, flags, and recommends — doesn't decide. This week's Copilot reads and reasons; Week 3 is where it starts writing back
- Refuses unauthorized queries — if the requester's Role or Marking eligibility doesn't allow it, the Copilot says so rather than quietly complying

14. Copilot Limitations & Escalation Paths

Escalation isn't a failure state to minimize — it's a designed outcome. The flow is: Copilot Response → Confidence Check → (if low confidence) Escalate → Human Review. A Copilot that escalates a genuinely low-confidence case is working correctly; one that guesses confidently instead is the actual failure. Design this path now, before Day 3, not as a fix after something goes wrong in front of a user.

15. Common Pitfalls

- Skipping straight to prompt-writing — without translating the requirement first, you're guessing at scope; revisit the translation step before touching a prompt
- Testing only with engineers — a Copilot that looks right to you may still be wrong to the domain expert who'll actually use it
- No escalation path until something breaks — design the low-confidence route now; retrofitting it after a bad answer reaches a user is too late
- Treating the prototype as the deliverable — a working demo and an operational solution are different bars; know which one you're building at each stage

16. Glossary

Term	Meaning
LLM Pipeline	A reusable, staged sequence (prepare, retrieve, assemble, act) that answers a family of questions, not just one
Context Pipeline	The retrieval stage that decides what's relevant to a specific question
Ontology-Aware Retrieval	Retrieval that follows Ontology Link Types and respects governance, not just keyword matching
AI FDE Workflow	The five-stage discipline: Requirement → Use Case → Prototype → Test → Operational Solution
Operational Solution	A governed, reliable capability ready for production use, as distinct from a working prototype
Escalation Path	The designed route for a low-confidence Copilot response to reach a human reviewer

17. Further Practice

Continue into Day 3, the self-paced lab where you build your batch's Copilot for real, following the exact AI FDE workflow from today: framing the operational requirement, translating it into a scoped use case, prototyping it, and designing its escalation path — grounded in your batch's own Week 1 Ontology.